Application Development - Network Programming and Patterns - 1

Table of Contents

Networking and Design Patterns

We take our steps towards writing applications that will start talking to other applications. In conjunction, this segment will introduce some basics around network programming.

Networking and Design Patterns

Why are design patterns integrated here?

This is reasonable question to ask because it may not be obvious the relevance here. The very general answer here is that networking introduces complexity that has to be managed and it is easy for it to be unmanageable.

Why Networking?

Inevitably, a large majority of applications are network enabled and even more-so in 2026. Given the comment regarding complexity, they do have their own set of problems that we have to solve and manage.

Consider cases like:

- Streaming Audio and Video (Radio, Something the user can select?)
- Multiplayer Video games (Is it Adversarial, Cooperative or something inbetween?)
- File-Sharing and Multi-Point transfers/assmbling
- Update and patching system
- Distributed and centralised networks.

There is a lot of application management and the domain does seem to embed itself into the networking setup to some degree.

There is no "one size fits all" category.

Why Design Patterns?

Software engineers have been building software for a long time, patterns have evolved and come about to enable ways we can structure our application, encode behaviour or make it easy to create, clone and compose objects.

Networking applications encompass all of these problems.

- Needing to Synchronise? Do you want eventual consistency, is it something that needs to be in lock step or could it be resolved later?

Maybe we want to encode **state**, have an objects that we will resolve at a later time when the data is ready or identify when we have gone out of sync.

- Need object replication? Consider how we can communicate fields of an object over a network and ensure correctness.

Rationale

This rationale for both of these topics is better at the beginning to give some weight to both domains.

It can be easy to compartmentalise design patterns as something never involving networks and simply as a structural thing.

It is easy to throw `TCP` at every problem and move on because for trivial problems it is easy to get an implementation. However as soon as deal with latency or need something that works in the browser, we should consider other options.

Server-Client Model

Out of the two models we will discuss, this one is one that forms the basis for the others.

We have two objects in our model.

- Server
- Client

A server is a process that will process data a client will send to it.

A client is a process that will interact with a server, normally sending data to it.

Protocols

While we will get more into the depth of our different layers, we want to outline that as part of any communication medium, we need a way that we cna commonly talk between devices.

This is where protocols come in, similar to your `stdin` protocol for a command line application, where you handle `ADD`, `UPDATE`, etc we would form similar rules to ensure that processes understand what the messages are.

Server-Client Model - Case-Study: IRC

As a simple case-study, IRC (Internet Relay Chat).

IRC follows the **Server-Client** model, where you have:

- IRC Server - This will have information on channels and users logged in
- IRC Client - This will be a user connected to the server, accessing channels.

The protocol will require client application to be able to emit messages to the server. These could involve `LIST`, `MSG`', `JOIN`, `CONNECT`, `USER` and many others.

These will be a query or a command to the server. The server is then able to send messages back.

Server-Client Model - Case-Study: IRC

Going back to *File and IO*, you will understand that we have a similar problem within networking where our **IO** can be either **text** or **binary**.

For IRC's case, it is a **text** protocol which makes it easy to work with. IRC is also line-buffered/oriented. This means that when a newline is read in by the client or server, it treats it as the message.

There are a lot of similarities between how the IRC protocol and standard input system works.

Server-Client Model - Case-Study: IRC

However, some of these decisions create some drawbacks.

If the protocol is line buffered, it means that the multi-line statements will need additional handling to work around this issue or require the client to treat the separation of the text as distinct messages themselves.

It is worth planning out your protocol, simulating and testing it prior to stabilising the ruleset as any major changes make need to undertake a major overhaul.

Open Systems Interconnection Model (OSI Model)

We've discussed a fairly abstract layer of our network system. However, we should dive into the OSI model that forms the basis of networks and how they interact.

The OSI model provides a reference model and outlines the abstraction layers of different networking components. This includes physical, data transmission, transmission structure, transmission control, data exchange, presentation of data and application usage.

For most things, application protocols are our main focus for either using the protocols or building them.

Layer 1 - Physical

Focused on how raw bits and symbols are transferred. Physical links that worked with this like RS232 which had explicit control over how much data it could receive by adjusting its rate.

However, ethernet controllers implement this mechanism and other applications such as Wifi, USB, and PCI-E maintain mechanisms to operate directly with a stream of bits.

Layer 2 - Data Link

After being able to handle a stream of data, the need of structure starts arising. This involves data-frames. It also outlines what kind of medium the interaction is occurring on and being able to control the processing of frames.

We have partially reached networking capabilities here, we have a way to identifying devices and controlling how we process the frames they send. We have started structuring the data from the first layer into something more reasonable.

Layer 3 - Network Layer

At this point, we start handling the idea of networks after being able to handle a stream of bits and capture the data frames.

The network layer is where **IP** sits. It provides the capability around routing and embeds ICMP here as well. This is used to provide a simple mechanism to sends and receive messages between devices.

IP addresses are associated with devices now, it also provides a mechanism to message devices a little more directly rather than a clunky broadcast mechanism.

*Remember! In the early days, there were funky network constructions such as Rings which had this 'pass the parcel' effect.

Layer 4 - Transport Layer

At this point, we can start building HTTP!

We will concern ourselves with this layer and above, the transport layer is used to outline how data flows.

- TCP - When we send a message to a device, we need a response back, there is an order to our messages!
- UDP - We don't care about 'connections', we just simply send data to a target and it can arrive out of order.

Layer 5 - Session

Each application or device needs to construct a session. In this case, we have what are known as **Sockets**. A socket is a mechanism to create a context for sending and receiving data, it establishes a link between two devices.

Given session, transport, and networking mechanism, we can use the information about the device we are connected to, how we want to interact with that device and the flow of packets/protocol and session it belongs to. Do note, a device could have more than one session open.

Layer 6 - Presentation

Data can be encoded in a variety of ways, as part of a protocol or a protocol layer, we need to understand how that data is interpreted. Consider the following.

- Data Compression - gzip and brotli
- Serialisation Format - XML, JSON
- Encoding such as *Base64*

The presentation layer is also relevant to a segment of message encryption where data is to be encrypted and decrypted and how to establish this. This includes TLS and SSL.

Layer 7 - Application

This is where HTTP lives!

The application layer here is built upon the rest, it also means that layers can be possibly swapped out for an alternative.

However! HTTP/HTTPS make assumptions on the transport layer they operate on and therefore had been in the past, restricted to only using TCP/IP while lately a different variant of HTTP(v3) has evolved to utilise UDP.

Socket Programming

Now we launch into the foray of our first network programming activity.

Within C#, you are able to access `Socket` in which you can specify it being a `TCP` socket and using the `IP` address family.

```
IPEndPoint endpoint = new IPEndPoint(address, port); //127.0.0.1, 9000

using Socket listener = new(
    endpoint.AddressFamily, // IP Address Family
    SocketType.Stream, // SocketType - Can be Stream for TCP, Dgram for UDP
    ProtocolType.Tcp); // ProtocolType - Specifies the protocol type - TCP or UDP or others
```

Socket Programming - Server Listener

As part of the server operations, we want to wait for connections to occur.

```
listener.Bind(endpoint); // Binds the Address information to the socket
```

After binding address information, the socket needs a connection buffer and start waiting for connections.

```
listener.Listen(MaxConnectionsInBuffer); // Connection buffer size

// other operations.

listener.Accept(); // Waits for a connection - Usually handled in a loop
```

Accept will normally **Block** until a connection is received. By default our IO operations are also defaulting to **blocking** reads and writes for simplicity.

Socket Programming - Client

For a client, we can perform a similar set of operations at the beginning but we are concerned about connecting to a server.

Similar to a `listener` socket we created prior, we set up things to be similar but we will rename this to `client`.

```
IPEndPoint endpoint = new IPEndPoint(address, port); //127.0.0.1, 9000

using Socket client = new(
    endpoint.AddressFamily, // IP Address Family
    SocketType.Stream, // SocketType - Can be Stream for TCP, Dgram for UDP
    ProtocolType.Tcp); // ProtocolType - Specifies the protocol type - TCP or UDP or others
```

Socket Programming - Client

We do not need to handle incoming connections, we just need to establish a connection. Simply calling `Connect` on the client socket should trigger a response from the server once the connection has been established and allow the client to connection and send messages to it.

```
client.Connect(endpoint); // Provide endpoint so it can connect, will block once established.
```

Socket Programming

For both client and server, we are then able to extract an IO stream that we can interact with. This allows us to send and receive data.

We are able to use `Receive` and `Send`. There are `Async` variants which we will look at next time.

We will need to provide a buffer for both `Receive` and `Send` so it is able to operate on this data and send it directly.

For **Text** based protocols, we will need to convert this encoding. Make sure you **use** the size received from `Receive` instead of always assuming it is the same size.

Socket Programming - Server Code

```
using Socket listener = new(
    ipEndPoint.AddressFamily,
    SocketType.Stream,
    ProtocolType.Tcp);

listener.Bind(ipEndPoint);
listener.Listen(100);

var handler = await listener.AcceptAsync();
while (true)
{
    // Receive message.
    var buffer = new byte[1_024];
    var received = await handler.ReceiveAsync(buffer, SocketFlags.None);
    var response = Encoding.UTF8.GetString(buffer, 0, received);

    var eom = "<|EOM|>";
    if (response.IndexOf(eom) > -1 /* is end of message */)
    {
        Console.WriteLine(
            $"Socket server received message: \"{response.Replace(eom, "")}\"");

        var ackMessage = "<|ACK|>";
        var echoBytes = Encoding.UTF8.GetBytes(ackMessage);
        await handler.SendAsync(echoBytes, 0);
        Console.WriteLine(
            $"Socket server sent acknowledgment: \"{ackMessage}\"");

        break;
    }
}
```

Socket Programming - Client Code

```
var ipEndPoint = new IPEndPoint(ipAddress, 13);

using TcpClient client = new();
await client.ConnectAsync(ipEndPoint);
await using NetworkStream stream = client.GetStream();

var buffer = new byte[1_024];
int received = await stream.ReadAsync(buffer);

var message = Encoding.UTF8.GetString(buffer, 0, received);
Console.WriteLine($"Message received: \"{message}\"");
```

State Pattern

Within network applications, the problem around **state** comes up a lot. As seeing some amount of this with an **ORM**, you can start considering why we want to encode state information into our objects.

However, **State** pattern is concerned with dictating how behaviour occurs on an object. Let's examine a scenario with an IRC server.

State Pattern - IRC Server, Managing Users and Channels

Within an IRC server, we have many users that will be connected and have joined channels. We will concern ourselves with the states around a user who is anonymous, registered and with channels that accept only registered users or allow anonymous users but will silence them.

We can encode state with users with a type **UserState** and for channels **ChannelState** (granted, another pattern like decorator could be used with channels that would be more applicable but we are keeping it simple.)

State Pattern - User States

We will have an interface that outlines if the user is anonymous or registered.

```
public interface UserState {
  IRCResponse Join(Channel channel, User user);

  // Other methods

  UserState Login(string credentials, User user, ServerContext context);
}
```

- `IRCResponse` is an object that is returned when a user attempts to join a channel. This is an action that the state can perform but not necessarily one that changes the state of the user.
- `Login` is a method that changes the state of the user, this applies to anonymous users or registered users. We will have two sub-types here. `AnonymousUserState`

```
public class AnonymousUserState : UserState {

  public IRCResponse Join(Channel channel, User user) {
    if(channel.RegisteredOnly) {
      return IRCResponse.OnlyRegisteredAllowed();
    } else {
      channel.register(user);
    }
  }

  public UserState Login(string credentials, User user, ServerContext context) {
    if(context.Login(credentials, user)) {
      return new RegisteredUserState();
    } else {
      return this;
    }
  }
}
```

Observe specifically the `Login` method which return either itself or a new `UserState` object.

Internally with a `User` that performs this action, the state update will look like the following.

```
state = state.Login(credentials, user, context);
```

When operating on the state, since it returns a new state or the current one, the state itself will correspond to something which has the same set of methods. This just upgrades the user to a registered user if valid and enables a different set of behaviour.

State and Strategy Pattern

There is a lot of similarity between state pattern and strategy, however the state pattern is concerned with mutating the state itself.

Both patterns focus on dependency injection, as in, using an interface and specifying the methods needed/it will work on rather than a concrete type.

Strategy pattern though, ensures that we operate on some context or we have some "Strategy" involved while state will change the object associated.

Proxy Pattern

The proxy pattern is a fairly broad idea here, It encompasses the idea of having a representation of an object that it does not entirely own or hasn't revealed the value yet as it is in transit.

Within IRC as a case study can be the idea of multiple segments to a message.

If we adhere to the limit of **512**bytes for a message with IRC and want to enable messages longer than this, we want to encode this idea. A proxy is suitable in this case as we can indicate to the server how many messages are being sent and the data size ahead of time.

The server can reserve the messages ahead of time and complete them once data is provided.

Proxy Pattern

There are some similarities to state pattern but generally, Proxy is focused on the lack of completeness or acting as an intermediary instead of the real thing. To demonstrate how this will work, lets look at the following.

```
public class MessageContainer {
  public byte[] Data { get; set; }
  public boolean Complete { get; set; }
  public MessageContainer() {}
}

public class CompleteMessage {

  public MessageContainer[] Messages { get; private set; }
  public CompleteMessage(IncompleteMessage messageToUpgrade) {
    // Takes containers
  }
}

public class IncompleteMessage {
  public MessageContainer[] Messages { get; private set; }
```

Proxy Pattern

Within the complete message class, we can see that it acts as a way to upgrade the message from
something that was previous incomplete. It will take the containers once the incomplete message has been satisfied.

```
public class CompleteMessage {

  public MessageContainer[] Messages { get; private set; }
  public CompleteMessage(IncompleteMessage messageToUpgrade) {
  }
}
```

Proxy Pattern

This is where there is more of a focus, IncompleteMessage has reserved the information but we have nothing to produce right now.

```
public class IncompleteMessage {
  public int MessageBucketIndex { get; private set; }
  public MessageContainer[] Messages { get; private set; }
  public IncompleteMessage(int nMessages) {
    Messages = new MessageContainer[nMessages]; // Initialise
  }

  public void FillMessage(byte[] data) {
    // populate a message[MessageBucketIndex]
  }

  public bool IsComplete() {
    // Returns to outline if the messages are complete or not
  }

  public CompleteMessage Upgrade() {
```

Proxy Pattern

The `Upgrade` method will be utilised when it is clear the proxy is no longer needed and we can move it to a complete message.

```
  public CompleteMessage Upgrade() {
    return new CompleteMessage(this);
  }
```